WIZ AI — Pipeline Architecture Session Summary

Date: 28 May 2026 **Project:** Wiz AI Music Video Platform **Checkpoint:** `fcdd423c`

Overview

This session completed a full two-phase architectural overhaul of the WIZ AI music video generation pipeline, implementing every item from the architecture specification document. The work spans from immediate prompt-engineering fixes through to a production-grade manifest-driven scene graph with webhook-first provider I/O, queue locking, and automated lip-sync quality gating.

Final test result: 802 tests pass across 67 test files. TypeScript: 0 errors.

Phase 1 — Immediate Code Changes

1a. Default Shot Mix: 75% → 80%

The performance-to-cinematic ratio default was changed from 75% to 80% across all four locations where it is set, consistent with the architecture specification’s recommendation for vocal-led tracks.

Location	Change
`server/music-video-service.ts`	`generateStoryboard` function signature default
`drizzle/schema.ts`	`musicVideoJobs.performanceShotRatio` column default
`server/routers/musicVideo.ts`	`createJob` tRPC DB insert default
`client/src/pages/MusicVideoAutopilot.tsx`	UI localStorage default

The DB migration was applied. The UI slider still allows users to set any value from 20% to 95%.

1b. Storyboard LLM Linting Rules

Seven new rules were injected into the storyboard LLM prompt:

1. **Vocal-only singing scenes** — only scenes with active lead vocals may be classified as singing performance scenes
2. **Minimum performance density** — at least four out of every five vocal scenes must be performance-led
3. **Maximum consecutive intercuts** — no more than two consecutive intercuts unless there is no active lead vocal
4. **Framing requirement** — at least half of all performance scenes must be medium close-up or head-and-shoulders framing
5. **Population and atmosphere enforcement** — every hall/studio prompt must explicitly include orchestra, audience, stage lighting, haze, room depth, and instruments when desired
6. **Forbidden prompt patterns** — grey background, isolated studio background, floating portrait, microphone-only hero frame, empty stage, duplicate singer, vague “performs on stage” language
7. **Structured prompt slots** — the LLM must populate: `shot_size`, `face_priority`, `environment`, `population`, `camera_motion`, `emotion`, `tempo_motion`, `forbidden_tokens`

1c. Raw Scene Validation Gate v2

The `server/raw-scene-validator.ts` was upgraded with five additional perception-level checks, bringing the total to seven:

Check	Description
Grey background	Rejects flat grey or low-variance backgrounds
Missing performer	Rejects scenes where a performer was required but absent
Head crop	Rejects scenes where the crown or chin is truncated
Face size	Rejects scenes where the face occupies too little of the frame
Missing population	Rejects scenes where orchestra/audience was required but absent
Weak framing	Rejects scenes with poor composition for a performance shot
Empty environment	Rejects scenes with an empty or featureless background

The heartbeat now auto-infers `requiresPopulation` from scene prompt keywords before calling the validator.

Phase 2 — Architecture

2a. Manifest Schema

Three new tables were added and applied via DB migration:

`renderManifests` — versioned JSON manifest per job: `jobId`, `version`, `manifestJson` (full scene graph, seeds, provider/model per scene, validation results, retry counters, final asset URLs), `status` enum (`draft`, `active`, `archived`).

`sceneAuditLog` — per-scene immutable audit trail: `sceneId`, `jobId`, `eventType`, `prompt`, `seed`, `providerModel`, `validationMetrics` (JSON), `screenshotUrl`, `notes`.

`sceneWebhookEvents` — idempotent webhook receipt log: `providerTaskId`, `eventType`, `payloadHash` (SHA-256), `processed`, unique constraint on

`(providerTaskId, eventType)` for deduplication.

2b. Webhook-First Provider I/O

New endpoint: `POST /api/webhooks/wavespeed` — registered before `express.json()` for raw body signature verification.

Full production webhook flow:

1. HMAC-SHA256 signature verification against `X-WaveSpeed-Signature` header
2. Deduplication via `sceneWebhookEvents` unique constraint
3. Scene lookup by `lipSyncTaskId` or `renderTaskId`
4. Immediate S3 copy on `completed` events (before any other processing)
5. State update: marks scene as `raw_generated` or `corrected` with permanent S3 URL
6. Failure handling: resets scene to `pending` for retry on `failed` events

---

2c. Immediate Asset Copy

Every WaveSpeed completion event triggers an immediate S3 copy before any downstream step. This protects against WaveSpeed's 7-day retention window. Provider URLs are never used as durable assets.

2d. SKIP LOCKED Queue + Advisory Locks

`server/queue-lock.ts` implements two locking mechanisms:

SKIP LOCKED queue helpers — prevent two concurrent heartbeat runs from claiming the same scene:

- `claimPendingScenes(db, jobId, limit)` — claims scenes for Seedance dispatch
- `claimPendingLipSyncScenes(db, jobId, limit)` — claims scenes for InfiniteTalk dispatch
- Both return empty arrays on error (fail-safe)

MySQL advisory locks — prevent double-assembly per job:

- `acquireJobLock(db, jobId, timeoutSeconds)` — named lock before triggering assembly
- `releaseJobLock(db, jobId)` — releases lock after assembly completes or fails
- `isJobLockFree(db, jobId)` — checks availability without acquiring

The assembly gate in `sceneDispatchHeartbeat.ts` wraps the assembly trigger in an advisory lock guard.

2e. LSE-D / LSE-C Lip-Sync Gate

`server/lip-sync-gate.ts` implements the full SyncNet-based acceptance thresholds.

Thresholds:

Gate	LSE-D	LSE-C	Action
GREEN	<= 8.0	>= 6.5	Proceed to assembly
AMBER	8.0-10.0	4.0-6.5	Proceed with audit flag
RED	> 10.0	< 4.0	Reset to pending, retry

Temporal alignment thresholds:

Result	Offset
Pass	<= 40 ms
Review	40-80 ms
Fail	> 80 ms

Two assessment modes:

1. **LLM vision assessment** (current deployment) — uses the built-in vision LLM to assess lip-sync quality from the video frame. Returns GREEN/AMBER/RED with reasoning. Runs in the Cloud Run Node.js runtime.

2. **SyncNet numeric metrics** (future, orchestration server) — when the $30 orchestration server is provisioned, it can run SyncNet inference and POST LSE-D/LSE-C values to a metrics endpoint. The gate then applies exact numeric thresholds with 0.95 confidence.

Integration: The gate is wired into the InfiniteTalk done handler. After InfiniteTalk completes and the video is copied to S3, the gate runs before `lipSyncStatus` is set to `done`. A RED result resets the scene to `pending` for retry. AMBER proceeds with a console warning.

New Files

File	Purpose
`server/lip-sync-gate.ts`	LSE-D/LSE-C threshold logic, LLM vision fallback, gate decision helpers
`server/queue-lock.ts`	SKIP LOCKED scene claiming, MySQL advisory locks
`server/webhooks/wavespeed.ts`	Webhook endpoint: HMAC-SHA256 verification, deduplication, S3 copy
`server/raw-scene-validator.ts`	Upgraded with 7 perception checks (v2)
`server/phase2-architecture.test.ts`	28 new tests for Phase 2 components

Test Summary

Test suite	Tests	Result
Phase 1 upgrades	18	All pass
Phase 2 architecture	28	All pass
Heartbeat tightening	Updated	All pass
Full suite	**802**	**All pass**

What Remains (Orchestration Server)

The following items require the $30 Standard cloud server (2 vCPU / 4 GB) and cannot run in the Cloud Run Node.js runtime:

- **SyncNet/Wav2Lip numeric LSE-D/LSE-C inference** — Python ML model, requires inference server
- **ffprobe media-level validation** — binary not available in Cloud Run
- **rembg background removal** — Python library, fallback compositing only
- **Persistent queue workers** — long-running background processes

The current deployment uses the LLM vision fallback for lip-sync gating and raw scene validation. Both are production-ready for the current scale. The numeric SyncNet path is fully implemented and will activate automatically when the orchestration server posts metrics to the webhook endpoint.

WIZ AI — Checkpoint `fcdd423c` *— 28 May 2026*